PRACTICAL 1:Vaccum Cleaner
Code:

```
class TableDrivenAgent:
    def __init__(self):
        self.table = {
            (('A', 'Dirty'),): 'Suck',
            (('A', 'Clean'),): 'Right',
            (('B', 'Dirty'),): 'Suck',
            (('B', 'Clean'),): 'Left',

            (('A', 'Clean'), ('B', 'Dirty')): 'Suck',
            (('A', 'Dirty'), ('B', 'Clean')): 'Right'
        }

        self.percepts = []

    def perceive(self, percept):
        self.percepts.append(percept)
        percept_sequence = tuple(self.percepts)
        action = self.table.get(percept_sequence, "No Action Found")
        return action

# Driver program
agent = TableDrivenAgent()
print(agent.perceive(('A', 'Dirty')))   # Suck

agent = TableDrivenAgent()
print(agent.perceive(('A', 'Clean')))   # Right

agent = TableDrivenAgent()
print(agent.perceive(('B', 'Dirty')))   # Suck

agent = TableDrivenAgent()
print(agent.perceive(('B', 'Clean')))   # Left
```

PRACTICAL 2: Peas Taxi
Code:

```
class TableDrivenAgent:
    def __init__(self, table):
        self.table = table

    def perceive_and_act(self, percept):
        return self.table.get(percept, "STOP_AND_WAIT")

decision_table = {
    ("road-clear", "green", "no-obstacle"): "MOVE_FORWARD",
    ("road-clear", "red", "no-obstacle"): "STOP",
    ("traffic", "green", "no-obstacle"): "SLOW_DOWN",
    ("road-clear", "green", "obstacle-ahead"): "TURN_LEFT",
    ("road-clear", "green", "pedestrian-detected"): "STOP",
    ("heavy-traffic", "green", "no-obstacle"): "REDUCE_SPEED",
}

agent = TableDrivenAgent(decision_table)

percepts = [
    ("road-clear", "green", "no-obstacle"),
    ("road-clear", "red", "no-obstacle"),
    ("traffic", "green", "no-obstacle"),
    ("road-clear", "green", "obstacle-ahead"),
    ("road-clear", "green", "pedestrian-detected"),
    ("unknown-space", "green", "no-obstacle"),
]

for p in percepts:
    action = agent.perceive_and_act(p)
    print(f"Percept: {p} -> Action: {action}")
```

PRACTICAL 3a: 8 Puzzle A*
Code:

```
import heapq

# Get the goal position of each tile
def get_goal_positions(goal):
    positions = {}
    for i, value in enumerate(goal):
        positions[value] = (i // 3, i % 3)
    return positions

# Manhattan Distance Heuristic
def manhattan(state, goal_positions):
    distance = 0

    for i, value in enumerate(state):
        if value == 0:
            continue

        x, y = i // 3, i % 3
        gx, gy = goal_positions[value]

        distance += abs(x - gx) + abs(y - gy)

    return distance

# Generate all neighboring states
def get_neighbors(state):
    neighbors = []
```

```
    blank = state.index(0)
    x, y = blank // 3, blank % 3

    moves = [
        ("Up", -1, 0),
        ("Down", 1, 0),
        ("Left", 0, -1),
        ("Right", 0, 1)
    ]

    for move, dx, dy in moves:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_blank = nx * 3 + ny
            new_state = list(state)

            new_state[blank], new_state[new_blank] =(
                new_state[new_blank],

                new_state[blank]
                )

            neighbors.append((tuple(new_state), move))

    return neighbors

# A* Search Algorithm
def astar(initial, goal):
    goal_positions = get_goal_positions(goal)

    priority_queue = []
    heapq.heappush(
        priority_queue,
        (manhattan(initial, goal_positions), 0, initial, [])
    )

    visited = {}

    while priority_queue:
        f, g, state, path = heapq.heappop(priority_queue)

        if state == goal:
            return path

        if state in visited and visited[state] <= g:
            continue

        visited[state] = g

        for neighbor, move in get_neighbors(state):
            new_g = g + 1
            h = manhattan(neighbor, goal_positions)
            new_f = new_g + h

            heapq.heappush(
                priority_queue,
                (new_f, new_g, neighbor, path + [move])
            )

    return None

# Example
initial = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

goal = (
    1, 0, 3,
    4, 5, 6,
    2, 7, 8
)

solution = astar(initial, goal)

if solution:
    print("A* Search Solution (Optimal Path):")
    print(" -> ".join(solution))
    print("Total Moves:", len(solution))
else:
    print("No solution found.")
```

Practical 3b : bfs 8 puzzle
Code:

```
from collections import deque

def find_blank(state):
    return state.index(0)

def get_neighbors(state):
    neighbors = []
    blank = find_blank(state)
```

```
    moves = {
        "Up": -3,
        "Down": 3,
        "Left": -1,
        "Right": 1
    }

    for move, step in moves.items():
        new_blank = blank + step

        if move == "Left" and blank % 3 == 0:
            continue

        if move == "Right" and blank % 3 == 2:
            continue

        if move == "Up" and blank < 3:
            continue

        if move == "Down" and blank > 5:
            continue

        new_state = list(state)

        new_state[blank], new_state[new_blank] = (
            new_state[new_blank],
            new_state[blank]
        )

        neighbors.append((tuple(new_state), move))

    return neighbors

def bfs(initial, goal):
    queue = deque([(initial, [])])
    visited = set()

    while queue:
        current_state, path = queue.popleft()

        if current_state == goal:
            return path

        if current_state in visited:
            continue

        visited.add(current_state)

        for next_state, move in get_neighbors(current_state):
            if next_state not in visited:
                queue.append((next_state, path + [move]))

    return None

initial_state = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

goal_state = (
    1, 2, 0,
    4, 5, 3,
    6, 7, 8
)

solution = bfs(initial_state, goal_state)

if solution:
    print("Shortest sequence of moves:")
    print(" -> ".join(solution))
    print("Number of moves:", len(solution))
else:
    print("No solution found.")
```

PRACTICAL 4: Water Jug

Code:

```
def dfs_water_jug(cap1, cap2, target):
    start = (0, 0)
    stack = [(start, [])]
    visited = set()

    def get_next_states(state):
        x, y = state
        next_states = []

        next_states.append(((cap1, y), "fill jug1"))

        next_states.append(((x, cap2), "fill jug2"))

        next_states.append(((0, y), "empty jug1"))

        next_states.append(((x, 0), "empty jug2"))

        pour = min(x, cap2 - y)
        next_states.append(((x - pour, y + pour), "pour jug1->jug2"))
```

```
    pour = min(y, cap1 - x)
    next_states.append(((x + pour, y - pour), "pour jug2->jug1"))

    return next_states

  while stack:
    state, path = stack.pop()

    if state in visited:
      continue

    visited.add(state)

    x, y = state

    if x == target or y == target:
      return path + [f"Reached target: {state}"]

    for next_state, action in get_next_states(state):
      if next_state not in visited:
        stack.append((next_state, path + [action]))

  return None

cap1,cap2=4,3
target = 2
result = dfs_water_jug(cap1, cap2, target)

if result:
  print("solution found using DFs:")
  for step in result:
   print(step)
else:
  print("no solution found:")
```